Chapter 11: Sets and Directories

Fundamentals of Python: Data Structures, Second Edition

Learning Objectives (1 of 2)

- Describe the features of a set and the operations on it
- Choose a set implementation based on its performance characteristics
- Recognize applications where it is appropriate to use a set
- Describe the features of a dictionary and the operations on it
- Choose a dictionary implementation based on its performance characteristics

Learning Objectives (2 of 2)

- Recognize applications where it is appropriate to use a dictionary
- Use a hashing strategy to implement unordered collections with constant-time performance

Using Sets (1 of 2)

- Set
  - A collection of items in no particular order
- In mathematics
  - You perform many operations on sets
- To describe the contents of a set,
  - You use the notation **`{<item-1> … <item-n>}`**
  - Assume the items are in no particular order

Using Sets (2 of 2)

- Table 11.1 Results of Some Typical Set of Operations

Set	Value	Union	Intersection	Difference	Subset
A	{12 5 17 6}	{12 5 42 17 6}	{17 6}	{12 5}	False
B	{42 17 6}				
A	{21 76 10 3 9}	{21 76 10 3 9}	{}	{21 76 10 3 9}	True
B	{}				
A	{87}	{22 87 23}	{87}	{}	False
B	{22 87 23}				
A	{22 87 23}	{22 87 23}	{87}	{22 23}	True
B	{87}				

The Python Set Class (1 of 2)

- Table 11.2 The Commonly Used Operations on the Set Type

Set Method	What It Does
`s = set()`	Creates an empty set and assigns it to `s`.
`s = set(anIterable)`	Creates a set that contains the unique items in `anIterable` object and assigns it to `s`.
`s.add(item)`	Adds `item` to `s` if it is not already in `s`.
`s.remove(item)`	Removes `item` from `s`.
`s.__len__()`	Same as `len (s)`. Returns the number of items currently in `s`.
`s.__iter__()`	Returns an iterator on `s`. Supports a `for loop` with `s`. Items are visited in an unspecified order.
`s.__str__()`	Same as `str(s)`. Returns a string containing the string representation of the items in `s`.

The Python Set Class (2 of 2)

- Table 11.2 The Commonly Used Operations on the Set Type (continued)

Set Method	What It Does
`s.__contains__(item)`	Same as item in `s`. Returns `True` if item is in `s`, or `False` otherwise.
`S1.__or__(s2)`	Set union. Same as `s1 | s2`. Returns a set containing items in `s1` and `s2`.
`S1.__and__(s2)`	Set intersection. Same as `s1 & s2`. Returns a set containing the items in `s1` that are also in `s2`.
`S1.__sub__(s2)`	Set difference. Same as `s1 - s2`. Returns a set containing the items in `s1` that are not in `s2`.
`S1.issubset(s2)`	Returns `True` if `s1` is a subset of `s2`, or `False` otherwise.

A Sample Session with Sets

- For example, create two sets named A and B and perform some operations on them:

```
>>> A = set([0, 1, 2])
>>> B = set()
>>> 1 in A
True
>>> A & B
{}
>>> B.add(1)
>>> B.add(1)
>>> B.add(5)
>>> B
{1, 5}
>>> A & B
{1}
>>> A | B
{0, 1, 2, 5}
>>> A - B
{0, 2}
>>> B.remove(5)
>>> B
{1}
>>> B.issubset(A)
True
  >>> for item in A:
        print(item, end = "")
0 1 2
```

Application of Sets

- Sets have many applications in the area of data processing
- For example, in the field of database management,
  - The answer to a query that contains the conjunction of two keys could be constructed from the intersection of the sets of items associated with those keys

Relationship between Sets and Bags

- Primary difference between sets and bags
  - Sets contain unique items
  - Bags can contain multiple instances of the same item
- A set type also includes operations
  - Such as intersection, difference, and subset

Relationship between Sets and Dictionaries

- Dictionary
  - An unordered collection of elements called entries
  - Each entry consists of a key and an associated value
  - Keys must be unique, but its values may be duplicated
  - Think of dictionary as having a set of keys

Implementation of Sets

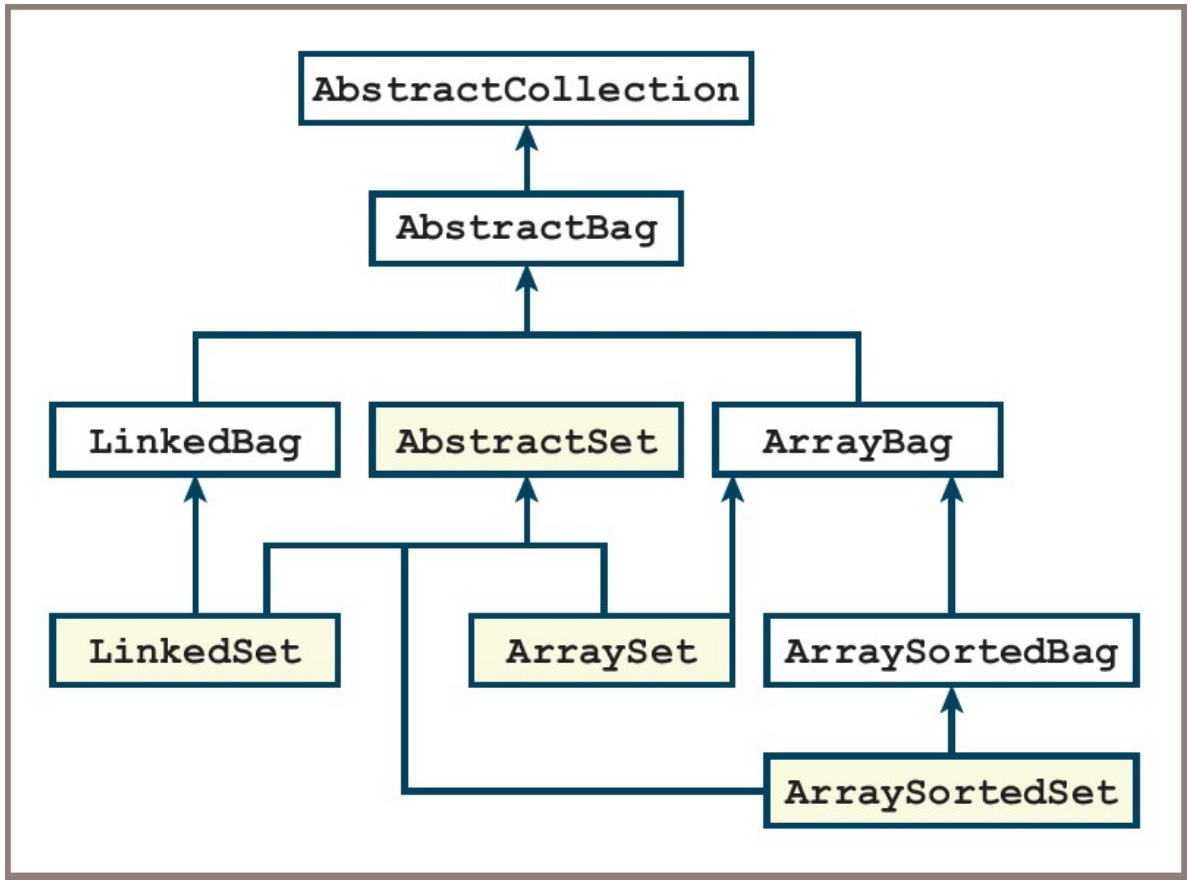
- Use arrays or linked structures to contain the data of items of a set
- Linked structure
  - Has an advantage of supporting constant-time removals of items once they are located in the structure
- Another strategy called *hashing*
  - Attempts to approximate random access into an array for insertions, removals, and searches

Array-Based and Linked Implementations of Sets (1 of 2)

- Set implementations
  - **`ArraySet`**, **`LinkedSet`**, and **`ArraySortedSet`** support the methods in the set interface
- Receive most of their code via inheritance from parent classes
  - **`ArrayBag`**, **`LinkedBag`**, and **`ArraySortedBag`**
- Set-specific methods **`_and_`**, **`_or_`**, **`_sub_`**, and **`issubset`**
  - Could be included in each set class
  - Only run other methods in the set interface, so they have the same code in all implementations

Array-Based and Linked Implementations of Sets (2 of 2)

Figure 11-1: Array-based and linked set implementations



The `AbstractSet` Class (1 of 2)

- Code for AbstractSet:

```
"""
File: abstractset.py
Author: Ken Lambert
"""

class AbstractSet(object):
"""Generic set method implementations."""

    # Accessor methods
    def __or__(self, other):
        """Returns the union of self and other."""
        return self + other

    def __and__(self, other):
        """Returns the intersection of self and other."""
        intersection = type(self)()
        for item in self:
            if item in other:
                intersection.add(item)
        return intersection
```

The `AbstractSet` Class (2 of 2)

- Code for **`AbstractSet`** (continued):

```
    def __sub__(self, other):
        """Returns the difference of self and other."""
        difference = type(self)()
        for item in self:
            if not item in other:
                difference.add(item)
        return difference

    def issubset(self, other):
        """Returns True if self is a subset of other
        or False otherwise."""
        for item in self:
            if not item in other:
                return False
        return True

    # The __eq__ method for sets goes here (exercise)
```

The `ArraySet` Class

- Code for **`ArraySet`**:

```
"""
File: arrayset.py
Author: Ken Lambert
"""

from arraybag import ArrayBag
from abstractset import AbstractSet

class ArraySet(AbstractSet, ArrayBag):
    """An array-based implementation of a set."""

def __init__(self, sourceCollection = None):
    ArrayBag.__init__(self, sourceCollection)

def add(self, item):
    """Adds item to the set if it is not in the set."""
    if not item in self:
        ArrayBag.add(self, item)
```

Using Dictionaries

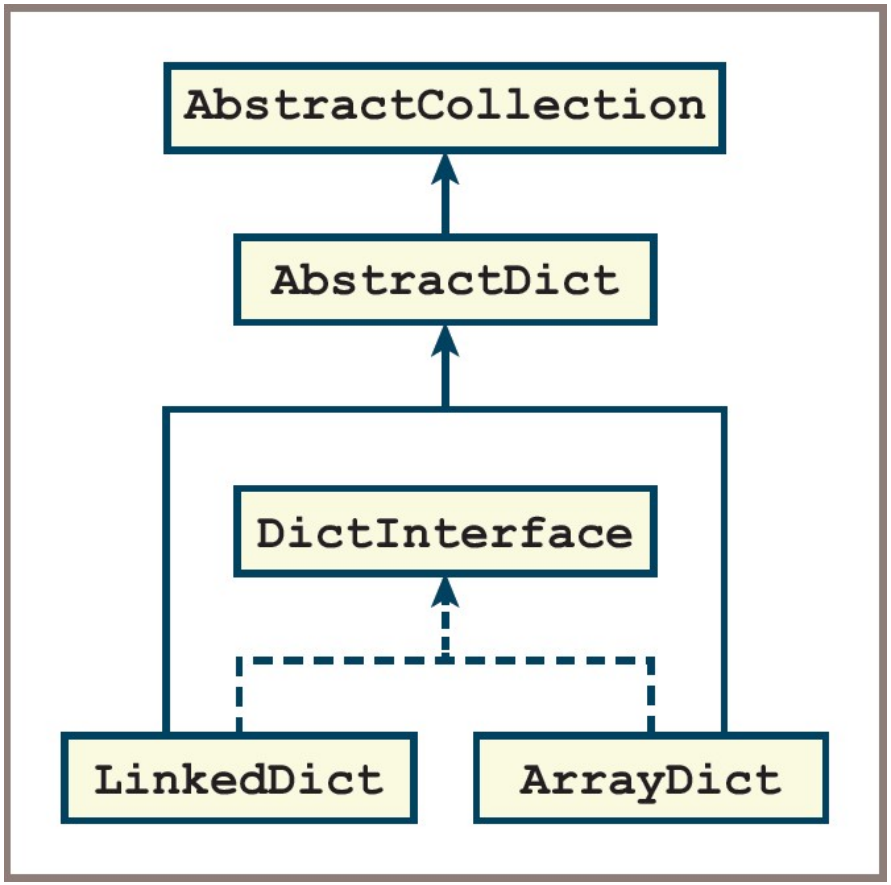
- **`dict`** type
  - Values are inserted or replaced at given keys using the subscript operator **`[ ]`**
- **`pop`** method
  - Removes a value at a given key
- **`keys`** and **`values`** methods
  - Return iterators on a dictionary's set of keys and collection of values
- **`_iter_`** method
  - Supports a **`for`** loop over a dictionary's keys
- **`get`** method
  - Allows you to access a value at a key or recovery by returning a default value if the key is not present

Array-Based and Linked Implementations of Dictionaries (1 of 4)

- Design strategy
  - Place the new classes in the collections framework so that they get some data and methods for free by means of inheritance from ancestor classes
  - If other methods in the new interface have the same implementation in all classes, place them in a new abstract class
- To achieve design goals,
  - Add an **`AbstractDict`** class to the framework as a subclass of **`AbstractCollection`**

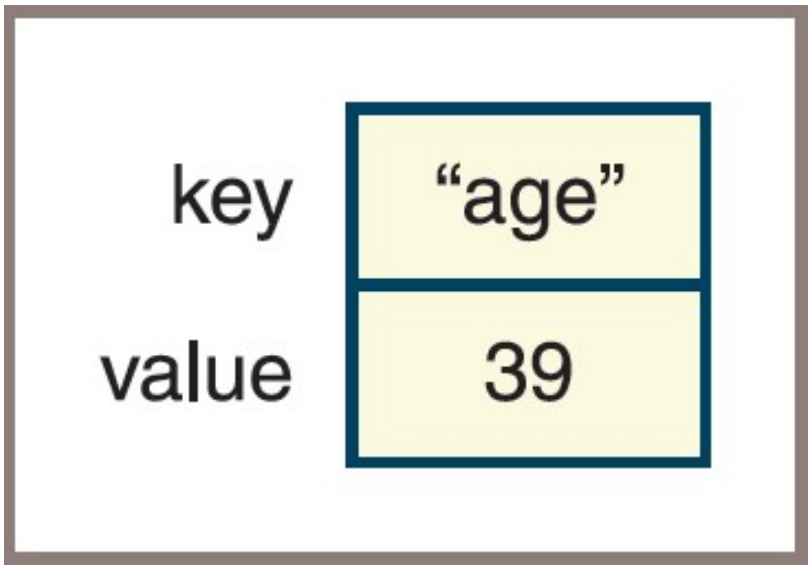
Array-Based and Linked Implementations of Dictionaries (2 of 4)

Figure 11-2: Array-based and linked dictionary implementations



Array-Based and Linked Implementations of Dictionaries (3 of 4)

Figure 11-3: An entry for a dictionary



Array-Based and Linked Implementations of Dictionaries (4 of 4)

- Code:

```
class Entry(object):
    """Represents a dictionary entry.
  Supports comparisons by key."""

    def __init__(self, key, value):
        self.key = key
        self.value = value

    def __str__(self):
        return str(self.key) + ":" + str(self.value)

    def __eq__(self, other):
        if type(self) != type(other): return False
        return self.key == other.key

    def __lt__(self, other):
        if type(self) != type(other): return False
        return self.key < other.key

    def __le__(self, other):
        if type(self) != type(other): return False
          return self.key <= other.key
```

The `AbstractDict` Class (1 of 3)

- The **`AbstractDict`** class includes all methods that call only other dictionary methods to do their work
  - Such as **`_str_`**, **`_add_`**, and **`_eq_`**
- The **`_init_`** method in **`AbstractDict`**
  - Does the work of copying keys and values from the optional source collections to the new dictionary object
  - Done after the **`_init_`** method in **`AbstractCollection`** is called with no collection argument

The `AbstractDict` Class (2 of 3)

- Code:

```
"""
File: abstractdict.py
Author: Ken Lambert
"""

from abstractcollection import AbstractCollection

class AbstractDict(AbstractCollection):
    """Common data and method implementations
    for dictionaries."""

    def __init__(self, keys, values):
        """Will copy entries to the dictionary
        from keys and values if they are present."""
        AbstractCollection.__init__(self)
        if keys and values:
            valuesIter = iter(values)
            for key in keys:
                self[key] = next(valuesIter)

    def __str__(self):
        return "{" + ", ".join(map(str, self.entries())) + "}"

    def __add__(self, other):
        """Returns a new dictionary containing the contents
        of self and other."""
        result = type(self)(self.keys(), self.values())
        for key in other:
            result[key] = other[key]
          return result
```

The `AbstractDict` Class (3 of 3)

- Code (continued):

```
def __eq__(self, other):
        """Returns True if self equals other,
        or False otherwise."""
        if self is other: return True
        if type(self) != type(other) or \
           len(self) != len(other):
            return False
        for key in self:
            if not key in other:
                return False
        return True

    def keys(self):
        """Returns a iterator on the keys in
        the dictionary."""
        return iter(self)

    def values(self):
        """Returns an iterator on the values in
        the dictionary."""
        return map(lambda key: self[key], self)

    def entries(self):
        """Returns an iterator on the entries in
        the dictionary."""
        return map(lambda key: Entry(key, self[key]), self)

    def get(self, key, defaultValue = None):
        """Returns the value associated with key if key is
        present, or defaultValue otherwise."""
        # Exercise
          return defaultValue
```

The `ArrayDict` Class (1 of 2)

- The ArrayDict Class is responsible for initializing the collection's container object
  - And implementing the methods that must directly access this container
- Code:

```
"""
File: arraydict.py
Author: Ken Lambert
"""

from abstractdict import AbstractDict, Entry

class ArrayDict(AbstractDict):
    """Represents an array-based dictionary."""

    def __init__(self, keys = None, values = None):
        """Will copy entries to the dictionary
        from keys and values if they are present."""
        self.items = list()
        AbstractDict.__init__(self, keys, values)

    # Accessors
    def __iter__(self):
        """Serves up the keys in the dictionary."""
        cursor = 0
        while cursor < len(self):
            yield self.items[cursor].key
            cursor += 1
```

The `ArrayDict` Class (2 of 2)

- Code (continued):

```
    def __getitem__(self, key):
        """Precondition: the key is in the dictionary.
        Raises: a KeyError if the key is not in the dictionary.
        Returns the value associated with the key."""
        index = self.getIndex(key)
        if index == -1:
            raise KeyError("Missing: " + str(key))
        return self.items[index].value

    # Mutators
    def __setitem__(self, key, value):
        """If the key is not in the dictionary, adds the key
        and value to it.
        Otherwise, replaces the old value with the new value."""
        index = self.getIndex(key)
        if index == -1:
            self.items.append(Entry(key, value))
            self.size += 1
        else:
            self.items[index].value = value

    def pop(self, key, defaultValue = None):
        """Removes the key and returns the associated value
        if the key is in the dictionary,
        or returns the default value otherwise."""
        index = self.getIndex(key)
        if index == -1:
            return defaultValue
        self.size -= 1
        return self.items.pop(index).value

    def getIndex(self, key):
        """Helper method for key search."""
        index = 0
        for nextKey in self:
            if nextKey == key:
                return index
            index += 1
        return -1
```

Complexity Analysis of the Array-Based and Linked Implementation of Sets and Dictionaries

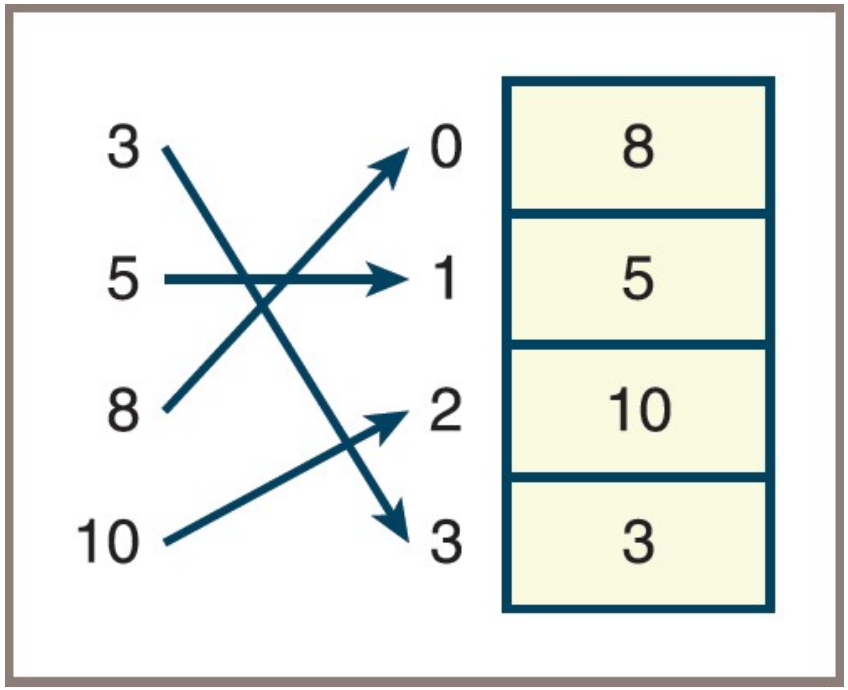
- Array-based implementations of sets and dictionaries require little programmer effort,
  - But, they do not perform well
- Each one must perform a linear search of the underlying array:
  - Each basic accessing method is O (*n*)

Hashing Strategies (1 of 3)

- Suppose the first key is 15,000:
  - Following keys are numbered consecutively
  - Position of a given key in an array could be compared with the expression `key - 15000`
- This type of computation is known as a *key-to-address transformation* or a *hashing function*
- The array used with a hashing strategy is called a hash table
- If the hashing function runs in constant time,
  - Insertions, accesses, and removals of the associated keys are O(1)

Hashing Strategies (2 of 3)

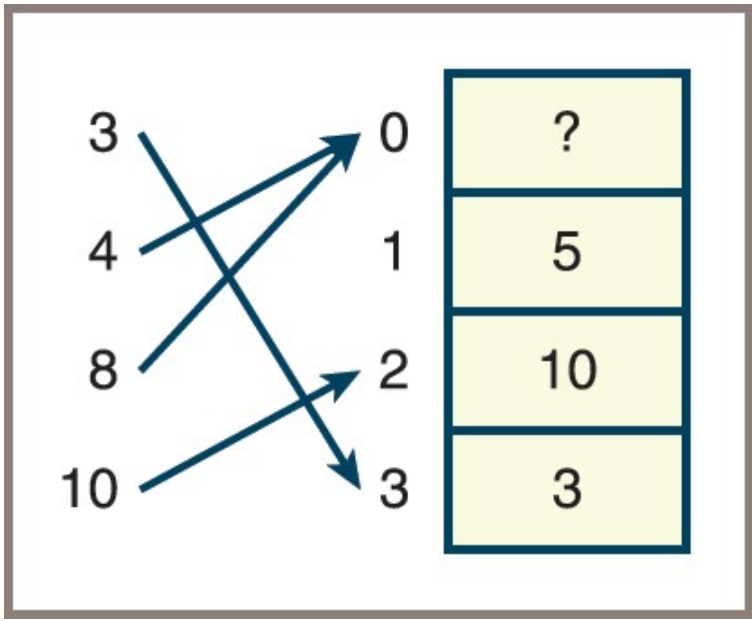
Figure 11-4: Placement of the keys 3, 5, 8, and 10 using the hashing function `key % 4`



Hashing Strategies (3 of 3)

- The hashing of the keys 4 and 8 to the same index is called a *collision*

Figure 11-5: Placement of the keys 3, 4, 8, and 10 using the hashing function `key % 4`



The Relationship of Collisions to Density (1 of 2)

- Do collisions occur when extra cells (beyond those needed for the data) are available in the array?
- To answer this question,
  - You'll write a Python function, **`keysToIndexes`**, which generates the indexes in an array of size *N* from a list of keys
- Definition of **`keysToIndexes`**:

```
def keysToIndexes(keys, n):
    """Returns the indexes corresponding to
    the keys for an array of length n."""
    return list(map(lambda key: key % n, keys))

>>> keysToIndexes([3, 5, 8, 10], 4)      # No collisions
[3, 1, 0, 2]
>>> keysToIndexes([3, 4, 8, 10], 4)      # One collision
  [3, 0, 0, 2]
```

The Relationship of Collisions to Density (2 of 2)

- Runs of both sets of keys with increasing array lengths show that no collisions occur when the array length reaches 8:

```
>>> keysToIndexes([3, 5, 8, 10], 8)
[3, 5, 0, 2]
>>> keysToIndexes([3, 4, 8, 10], 8)
[3, 4, 0, 2]
```

Hashing with Nonnumeric Keys

- Definition of **`stringHash`** function followed by a demonstration of how it handles anagrams:

```
def stringHash(item):
    """Generates an integer key from a string."""
      if len(item) > 4 and \
     (item[0].islower() or item[0].isupper()):
     item = item[1:]    # Drop first letter
    total = 0
    for ch in item:
        total += ord(ch)
      if len(item) > 2:
        total -= 2 * ord(item[-1]) # Subtract last ASCII
    return total

>>> stringHash("cinema")
328
>>> stringHash("iceman")
296
```

Linear Probing (1 of 4)

- For insertions
  - Simplest way to resolve a collision is to search the array, starting from the collision spot, for the first available position
  - Process is referred to as *linear probing*
- A position is considered to be available for the insertion of a key if it has never been occupied or if a key has been deleted from it:
  - The values **`EMPTY`** and **`DELETED`** designate these two states
- At the start of an insertion, the hashing function is run to compute the *home index* of the item:
  - The home index is the position where the item should go if the hash function works perfectly
  - If the cell at the home index is not available, the algorithm moves the index to the right to probe for an available cell
  - When the search reaches the last position of the array, the probing wraps around to continue from the first position

Linear Probing (2 of 4)

- Code for insertions into an array named table:

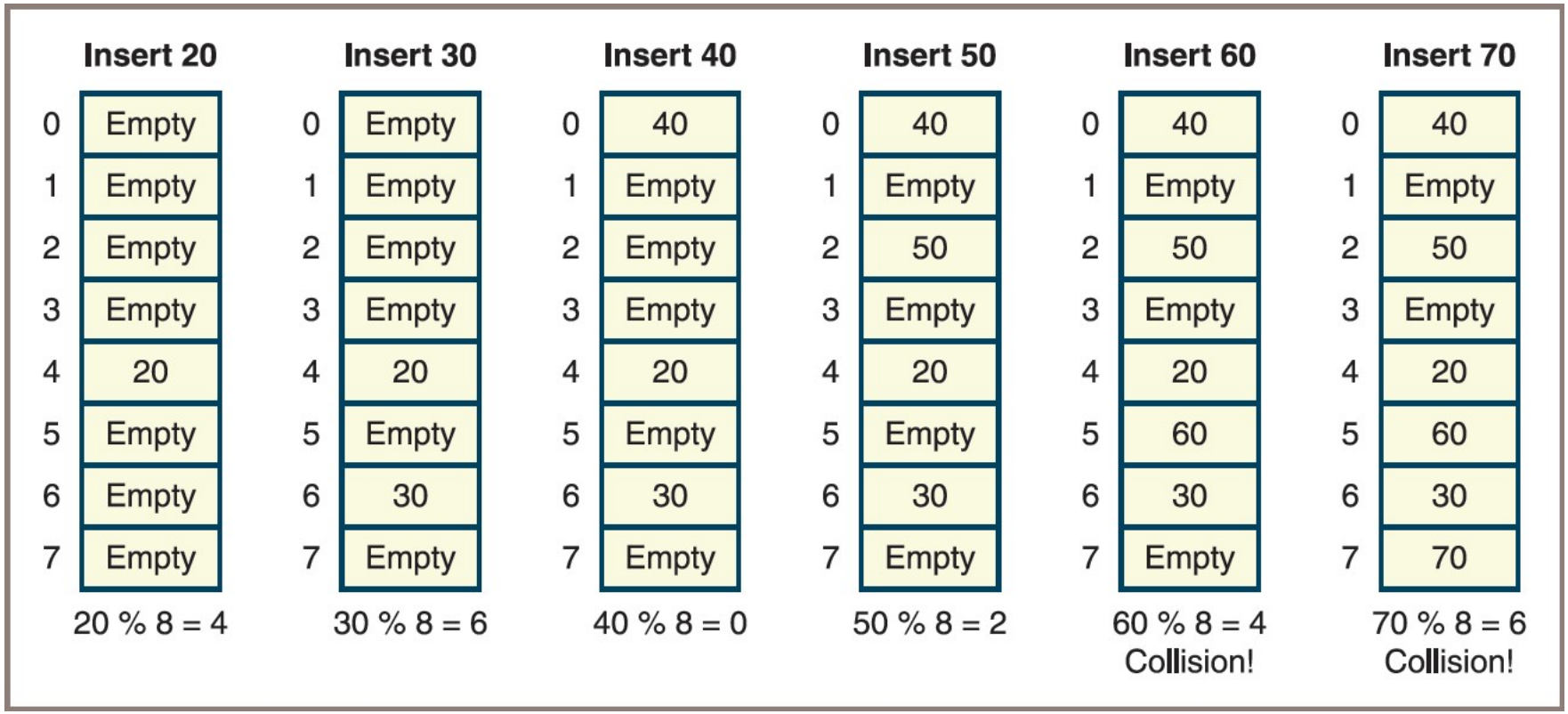
```
# Get the home index
index = abs(hash(item)) % len(table)
# Stop searching when an empty cell is encountered
while not table[index] in (EMPTY, DELETED):
    # Increment the index and wrap around to first
    # position if necessary
    index = (index + 1) % len(table)
# An empty cell is found, so store the item
table[index] = item
```

Linear Probing (3 of 4)

- Problems with resolving collisions
  - After several insertions and removals, a number of cells marked DELETED may lie between a given item and its home index
  - When the items that cause a collision are relocated in the same region (a cluster) within the array, it is known as *clustering*
- Clustering usually leads to collisions with other relocated items

Linear Probing (4 of 4)

Figure 11-6: Clustering during linear probing



Quadratic Probing (1 of 2)

- Way to avoid the clustering associated with linear probing
  - To advance the search for an empty position a considerable distance from the collision point
- *Quadratic probing* accomplishes this by incrementing the home index by the square of a distance on each attempt:
  - If you begin with home index k and a distance d, the formula used on each pass is $k + d^2$

Quadratic Probing (2 of 2)

- Code for insertions, updated to use quadratic probing:

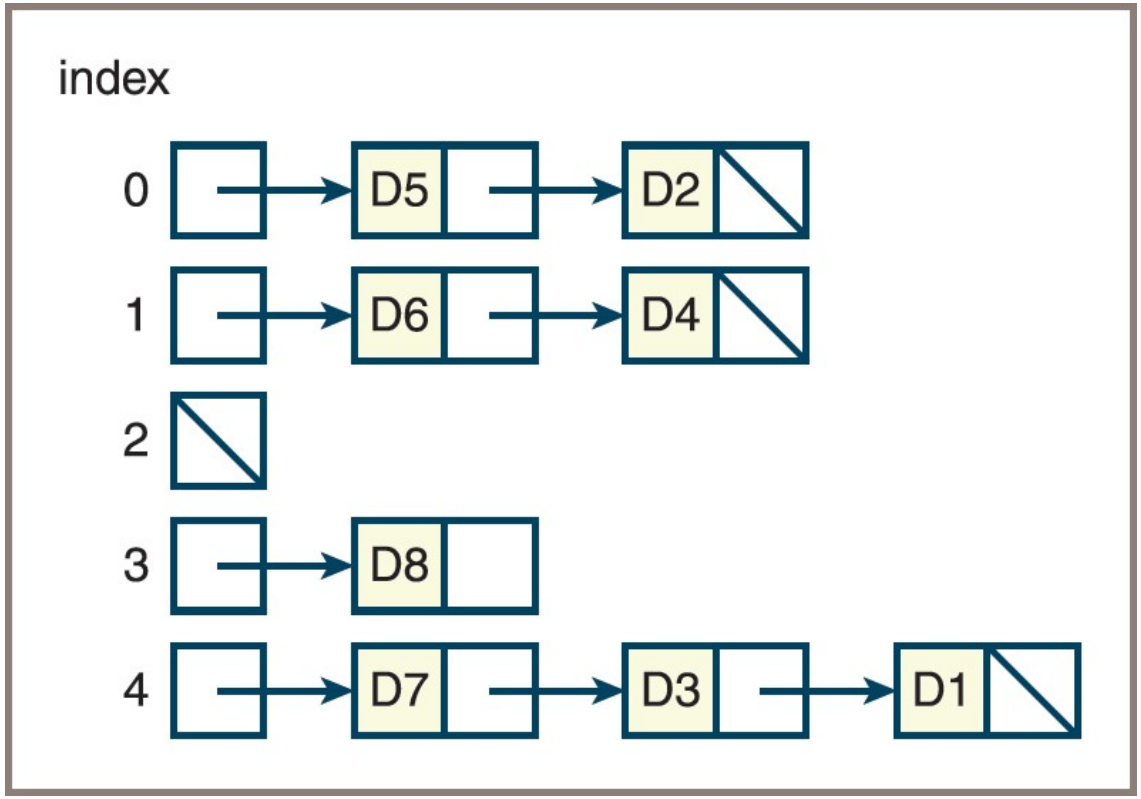
```
# Set the initial key, index, and distance
key = abs(hash(item))
distance = 1
homeIndex = key % len(table)
index = homeIndex
# Stop searching when an unoccupied cell is
encountered
while not table[index] in (EMPTY, DELETED):
    # Increment the index and wrap around to the
    # first position if necessary
    index = (homeIndex + distance ** 2) % len(table)
    distance += 1
# An empty cell is found, so store the item
table[index] = item
```

Chaining (1 of 3)

- Chaining
  - A collision-processing strategy where the items are stored in an array of linked lists, or *chains*
  - Each item's key locates the *bucket*, or index, of the chain in which the item already resides or is to be inserted
- The retrieval and removal operations perform the following:
  - Compute the item's home index in the array
  - Search the linked list at that index for the item

Chaining (2 of 3)

Figure 11-7: Chaining with five buckets



Chaining (3 of 3)

- Code for inserting an item using chaining:

```
# Get the home
index = abs(hash(item)) % len(table)
# Access a bucket and store the item at the head
# of its linked list
table[index] = Node(item, table[index])
```

Complexity Analysis (1 of 2)

- Complexity of linear collision processing
  - Depends on the load factor and the tendency of relocated items to cluster
- Because the quadratic method tends to mitigate clustering, you can expect its average performance to be better than that of the linear method
- According to Knuth (cited in text), the average search complexity for the quadratic method is

$$1 - \log_e(1-D) - \left(\frac{D}{2}\right)$$

for the successful case and

$$\frac{1}{(1-D)} - D - \log_e(1-D)$$

for the unsuccessful case

Complexity Analysis (2 of 2)

- Analysis of the bucket/chaining method shows that the process of locating an item consists of two parts:
  - Computing the home index
  - Searching a linked list when collisions occur

Hashing Implementation of Sets (1 of 6)

- The hashing implementation of a set is called **HashSet**:
  - It uses the bucket/chaining strategy
  - The implementation must maintain an array and represent entries in such a manner as to allow chaining
- To manage the array, you include three instance variables: **`items`** (the array), **`size`** (the number of items in the set), and **`capacity`** (the number of cells in the array)
  - The value of **`capacity`** is by default a constant, which is defined as 3 to ensure frequent collisions

Hashing Implementation of Sets (2 of 6)

- Table 11.6 The Variables Used for Accessing Entries in the Class **`HashSet`**

Instance Variable	Purpose
`self.foundNode`	Refers to the node just located, or is `None` otherwise.
`self.priorNode`	Refers to the node prior to the one just located, or is `None` otherwise.
`self.index`	Refers to the index of the chain in which the node was just located, or is −1 otherwise.

Hashing Implementation of Sets (3 of 6)

- Pseudocode for how _contains_ locates a node's position and sets variables:

```
__contains__ (item)
    Set index to the hash code of the item
    Set priorNode to None
    Set foundNode to table[index]
    while foundNode != None
        if foundNode.data == item
            return true
        else
            Set priorNode to foundNode
            Set foundNode to foundNode.next
  return false
```

Hashing Implementation of Sets (4 of 6)

- Partial implementation of the class **`HashSet`**:

```
from node import Node
from arrays import Array
from abstractset import AbstractSet
from abstractcollection import AbstractCollection

class HashSet(AbstractSet, AbstractCollection):
    """A hashing implementation of a set."""

    DEFAULT_CAPACITY = 3

    def __init__(self, sourceCollection = None,
                 capacity = None):
        if capacity is None:
            self.capacity = HashSet.DEFAULT_CAPACITY
        else:
            self.capacity = capacity
        self.items = Array(self.capacity)
        self.foundNode = self.priorNode = None
        self.index = -1
        AbstractCollection.__init__(self, sourceCollection)
```

Hashing Implementation of Sets (5 of 6)

- Partial implementation of the class **`HashSet`** (continued):

```
# Accessor methods
    def __contains__(self, item):
        """Returns True if item is in the set or
        False otherwise."""
        self.index = abs(hash(item)) % len(self.items)
        self.priorNode = None
        self.foundNode = self.items[self.index]
        while self.foundNode != None:
            if self.foundNode.data == item:
                return True
            else:
                self.priorNode = self.foundNode
                self.foundNode = self.foundNode.next
        return False

    def __iter__(self):
        """Supports iteration over a view of self."""
        # Exercise

    def __str__(self):
        """Returns the string representation of self."""
        # Exercise
```

Hashing Implementation of Sets (6 of 6)

- Partial implementation of the class **`HashSet`** (continued):

```
# Mutator methods
    def clear(self):
        """Makes self become empty."""
        self.size = 0
        self.foundNode = self.priorNode = None
        self.index = -1
        self.array = Array(HashSet.DEFAULT_CAPACITY)

    def add(self, item):
        """Adds item to the set if it is not in the set."""
        if not item in self:
            newNode = Node(item,
                           self.items[self.index])
            self.items[self.index] = newNode
            self.size += 1

    def remove(self, item):
        """Precondition: item is in self.
        Raises: KeyError if item in not in self.
        Postcondition: item is removed from self."""
        # Exercise
```

Hashing Implementation of Dictionaries (1 of 4)

- The hashing implementation of a dictionary is called **`HashDict`**:
  - It uses a bucket/chaining strategy (similar to **`HashSet`**)
- To represent a key/value entry, you use the **`Entry`** class defined earlier in the other implementations:
  - The **`data`** field of each node in a chain now contains an **`Entry`** object
- The **`__contains__`** method now looks for a key in the underlying structure and updates the pointer variables

Hashing Implementation of Dictionaries (2 of 4)

- The method **`__getitem__`** simply calls **`__contains__`** and returns the value contained in **`foundNode.data`** if the key was found:

```
__getitem__(key)
if key in self
return foundNodedata.value
else
raise KeyError
```

Hashing Implementation of Dictionaries (3 of 4)

- The method `__setitem__` calls `__contains__` to determine whether or not `an` entry exists at the target key's position:
  - If the entry is found, `__setitem__` replaces its value with the new value
- Otherwise, `__setitem__` performs the following steps:
  - Creates a new item object containing the key and value
  - Creates a new node whose data is the item and whose next pointer is the node at the head of the chain
  - Sets the head of the chain to the new node
  - Increments the size

Hashing Implementation of Dictionaries (4 of 4)

- Pseudocode for **`_setitem_`**:

```
__setitem__(key, value)
    if key in self
      foundNode.data.value = value
    else
      newNode = Node(Item(key, value), items[index])
      items[index] = newNode
  size = size + 1
```

Sorted Sets and Dictionaries (1 of 3)

- Sorted set and sorted dictionary
  - Have the behaviors of a set and a dictionary
  - But the user can visit their data in sorted order
- Each item added to a sorted set must be comparable with its other items
  - And each key added to a sorted dictionary must be comparable with its other keys
- The iterator for each type of collection guarantees its users access to the items or the keys in sorted order

Sorted Sets and Dictionaries (2 of 3)

- A common implementation of sorted sets uses a binary search tree:
  - Sorted sets and sorted dictionaries that use a tree-based implementation can provide logarithmic access to data items
- There are two design strategies for using a binary search tree in a sorted set implementation:
  - One strategy is to develop a tree sorted bag class that contains a binary search tree for its data items, and calls methods on that tree
  - Second strategy uses the **`LinkedBST`** class in a partially defined sorted set class called **`TreeSortedSet`** (see code on next slide)

Sorted Sets and Dictionaries (3 of 3)

```
from linkedbst import LinkedBST
from abstractCollection import AbstractCollection
from abstractset import AbstractSet

class TreeSortedSet(AbstractSet):
    """A tree-based implementation of a sorted set."""

  def __init__(self, sourceCollection = None):
        self.items = LinkedBST()
        if sourceCollection:
            for item in sourceCollection:
                self.add(item)

    def __contains__(self, item):
        """Returns True if item is in the set or
        False otherwise."""
        return item in self.items

    def __iter__(self):
        """Supports an inorder traversal on a view of self."""
        return self.items.inorder()

    def add(self, item):
        """Adds item to the set if it is not in the set."""
        if not item in self:
            self.items.add(item)

    # Remaining methods are exercises
```

Chapter Summary (1 of 2)

- A set is an unordered collection of items
- A list-based implementation of a set supports linear-time access:
  - A hashing implementation of a set supports constant-time access
- The items in a sorted set can be visited in sorted order:
  - A tree-based implementation of a sorted set supports logarithmic-time access
- A dictionary is an unordered collection of entries, where each entry consists of a key and a value:
  - Each key in a dictionary is unique, but its values may be duplicated
- A sorted dictionary imposes an ordering by comparison on its keys

Chapter Summary (2 of 2)

- Implementations of both types of dictionaries are similar to those of sets
- Hashing is a technique for locating an item in constant time:
  - This technique uses a hash function to compute the index of an item in an array
- When using hashing, the position of a new item can collide with the position of an item already in an array
- Chaining employs an array of buckets, which are linked structures that contain the items
- The run-time and memory aspects of hashing methods involve the load factor of the array